



ESPRIT
Cloud et Sécurité

Rapport de Stage

aws-iam-monitor

Conception et développement d'une plateforme serverless de supervision,
d'audit et de détection des menaces liées aux modifications AWS IAM

Réalisé par :	Zouari Omar
Organisme d'accueil :	Vermeg
Encadrant professionnel :	Mr.Amine Chtioui
Période de stage :	1 ^{er} juillet 2026 – 31 août 2026
Année universitaire :	1 ^{re} année cycle ingénieur
Dépôt du projet :	https://github.com/ZouariOmar/ aws-iam-monitor

Rapport rédigé à l'issue d'un stage consacré à la conception et à l'implémentation d'une architecture de sécurité AWS pour la supervision en temps réel des changements IAM, selon deux approches équivalentes : Bash/AWS CLI et Terraform.

Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de ce stage.

Mes remerciements s'adressent en premier lieu à Mr. Amine Chtioui, mon encadrant professionnel, pour son accueil, sa disponibilité et les conseils prodigués tout au long de cette période, ainsi qu'à l'ensemble de l'équipe de Vermeg pour le cadre de travail mis à ma disposition.

Enfin, je remercie ESPRIT et l'ensemble du corps enseignant de la formation Cloud et Sécurité pour les connaissances transmises tout au long de mon parcours, qui m'ont permis d'aborder ce stage dans de bonnes conditions.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage de deux mois portant sur la sécurité des environnements AWS, et plus particulièrement sur la supervision des changements apportés à la gestion des identités et des accès (*IAM – Identity and Access Management*). Une mauvaise configuration ou une modification malveillante des permissions IAM figure en effet parmi les causes les plus fréquentes d'incidents de sécurité dans le cloud.

Le stage s'est articulé autour du projet `aws-iam-monitor`, une plateforme serverless permettant de détecter, classer et notifier en temps quasi réel les opérations IAM sensibles effectuées sur un compte AWS. L'architecture retenue s'appuie sur AWS CloudTrail pour la capture des événements, Amazon EventBridge pour leur filtrage, AWS Lambda pour leur analyse et leur classification, puis Amazon SNS, Amazon S3 et Amazon CloudWatch respectivement pour la notification, l'archivage et la supervision.

Le projet a été développé selon deux implémentations indépendantes mais fonctionnellement équivalentes : une implémentation impérative en Bash/AWS CLI, pilotée par un outil en ligne de commande unique (`awsctl`), et une implémentation déclarative en Terraform, structurée en modules réutilisables. Cette double implémentation a permis de comparer concrètement les approches impérative et déclarative de l'infrastructure as code, chacune présentant des avantages et des limites propres.

Ce stage a été l'occasion d'approfondir des compétences en sécurité cloud, en architecture serverless, en scripting Bash, en infrastructure as code avec Terraform, ainsi qu'en développement Python pour le traitement d'événements. Il a également permis de développer une démarche rigoureuse de conception, de documentation et de validation d'une solution technique complète.

Mots-clés : AWS, IAM, sécurité cloud, CloudTrail, EventBridge, Lambda, SNS, Terraform, Infrastructure as Code, DevSecOps.

Abstract

This report presents the work carried out during a two-month internship focused on AWS security, specifically on monitoring changes made to Identity and Access Management (IAM). Misconfigured or maliciously altered IAM permissions remain among the most common causes of cloud security incidents.

The internship centered on `aws-iam-monitor`, a serverless platform designed to detect, classify, and notify sensitive IAM operations performed on an AWS account in near real time. The architecture relies on AWS CloudTrail to capture management events, Amazon EventBridge to filter them, AWS Lambda to analyze and classify them, and Amazon SNS, Amazon S3, and Amazon CloudWatch for alerting, archiving, and observability respectively.

The system was built as two independent, functionally equivalent implementations: an imperative Bash/AWS CLI implementation driven by a single command-line tool (`awsctl`), and a declarative Terraform implementation organized into reusable modules. This dual implementation made it possible to directly compare imperative and declarative infrastructure-as-code approaches, each with its own strengths and trade-offs.

Keywords: AWS, IAM, cloud security, CloudTrail, EventBridge, Lambda, SNS, Terraform, Infrastructure as Code, DevSecOps.

Table des matières

Remerciements	i
Résumé	ii
Introduction générale	1
1 Contexte général et étude préalable	2
1.1 Présentation de l'environnement du stage	2
1.2 Introduction au Cloud Computing et à AWS	2
1.3 AWS IAM et la problématique de la sécurité des identités	2
1.4 Limites et risques d'une mauvaise gestion IAM	3
1.5 Besoin d'une solution de supervision	3
2 Étude et conception de la solution	4
2.1 Problématique et objectifs	4
2.1.1 Objectifs fonctionnels	4
2.1.2 Objectifs techniques	4
2.2 Analyse des besoins et architecture générale	4
2.3 Choix des services AWS	6
2.4 Sécurité de l'architecture	6
2.5 Classification des risques	6
2.6 Stratégie de stockage et stratégie d'alerte	7
2.7 Infrastructure as Code : une conception à double implémentation	7
3 Réalisation	8
3.1 Structure générale du dépôt	8
3.2 IAM	8
3.3 CloudTrail	9
3.4 EventBridge	9
3.5 Lambda	9
3.6 SNS	9
3.7 S3	10
3.8 CloudWatch	10
3.9 Bash / AWS CLI – l'outil awsctl	10
3.10 Terraform	10
3.11 Comparaison Bash / Terraform	11
3.12 Tests et validation	11
4 Déploiement, validation et résultats	12
4.1 Prérequis	12
4.2 Installation et provisioning	12
4.2.1 Implémentation Bash	12
4.2.2 Implémentation Terraform	12
4.3 Déroulement d'un déploiement complet	12
4.4 Fonctionnement observé	13
4.4.1 Illustrations	13

4.5	Avantages de l'approche idempotente	13
4.6	Limites de la validation effectuée	14
5	Bilan et perspectives	16
5.1	Compétences techniques acquises	16
5.1.1	Compétences en Cloud AWS et sécurité	16
5.1.2	Compétences DevOps et Infrastructure as Code	16
5.1.3	Compétences en développement	16
5.1.4	Documentation et autonomie	16
5.2	Difficultés rencontrées	16
5.3	Limites actuelles du projet	17
5.4	Perspectives d'évolution	17
	Conclusion générale	18
	Points clés pour la soutenance	19

Introduction générale

Le passage massif des systèmes d'information vers le cloud computing a profondément transformé la manière dont les organisations conçoivent, déploient et sécurisent leurs infrastructures. Ce mouvement, porté par des fournisseurs comme Amazon Web Services (AWS), Microsoft Azure ou Google Cloud Platform, offre une élasticité et une rapidité de déploiement inédites, mais déplace également la surface d'attaque vers de nouveaux points critiques : la configuration des identités et des permissions.

Au sein d’AWS, le service *Identity and Access Management* (IAM) constitue le socle sur lequel repose l'ensemble du contrôle d'accès à un compte. Une politique IAM trop permissive, un utilisateur créé sans autorisation, ou une désactivation de l'authentification multifacteur (MFA) peuvent ouvrir la voie à des compromissions majeures. Contrairement aux vulnérabilités applicatives classiques, ces incidents ne relèvent pas d'une faille de code mais d'une mauvaise gestion des droits, ce qui les rend à la fois plus difficiles à détecter et potentiellement plus dommageables.

C'est dans ce contexte que s'inscrit le stage présenté dans ce rapport, effectué au sein de Vermeg du 1^{er} juillet au 31 août 2026. L'objectif du stage était de concevoir et développer une solution de supervision permettant de détecter en temps quasi réel les modifications sensibles apportées à IAM, de les classer selon leur niveau de risque, et de déclencher les alertes appropriées, tout en conservant une trace exploitable de chaque événement.

La problématique centrale de ce travail peut être formulée ainsi : *comment mettre en place, à moindre coût opérationnel et de façon entièrement automatisée, une supervision continue des changements IAM sur un compte AWS, capable de distinguer une activité légitime d'une activité potentiellement malveillante ?*

Pour y répondre, le projet `aws-iam-monitor` a été développé selon une architecture serverless événementielle, s'appuyant sur les services AWS CloudTrail, EventBridge, Lambda, SNS, S3 et CloudWatch. Deux implémentations indépendantes et fonctionnellement équivalentes ont été réalisées : l'une impérative, en Bash et AWS CLI, l'autre déclarative, en Terraform, ce qui a permis d'aborder à la fois le scripting d'infrastructure et l'infrastructure as code moderne.

La méthodologie suivie a consisté, dans un premier temps, en une phase d'apprentissage des services AWS impliqués, suivie d'une phase d'analyse du besoin et de conception de l'architecture, puis d'une phase de développement progressif de chaque composant, module par module, pour chacune des deux implémentations. Une dernière phase a été consacrée à la documentation du projet et à la préparation des livrables.

Ce rapport est organisé en cinq chapitres. Le premier chapitre présente le contexte général du stage et l'étude préalable de la problématique de sécurité IAM. Le deuxième chapitre détaille l'étude et la conception de la solution retenue. Le troisième chapitre décrit la réalisation technique du projet, composant par composant. Le quatrième chapitre aborde le déploiement, la validation et les résultats obtenus. Enfin, le cinquième chapitre dresse un bilan des compétences acquises et propose des perspectives d'évolution pour le projet.

Chapitre 1

Contexte général et étude préalable

1.1 Présentation de l'environnement du stage

Le stage s'est déroulé au sein de Vermeg, sur une période de deux mois, du 1^{er} juillet au 31 août 2026. Le travail a porté sur un projet technique concret, `aws-iam-monitor`, dont le développement constitue le fil conducteur de ce rapport. L'environnement de travail reposait sur un compte AWS de test, l'AWS CLI, un environnement Python 3.11, ainsi que les outils Bash et Terraform utilisés pour les deux implémentations du projet.

1.2 Introduction au Cloud Computing et à AWS

Le cloud computing consiste à mettre à disposition, à la demande et via Internet, des ressources informatiques (calcul, stockage, réseau, services managés) sans que l'utilisateur ait à gérer directement l'infrastructure physique sous-jacente. Amazon Web Services (AWS), l'un des principaux fournisseurs de cloud public, propose un catalogue de plusieurs centaines de services couvrant le calcul (EC2, Lambda), le stockage (S3), les bases de données, le réseau, la sécurité et bien d'autres domaines.

Ce modèle présente des avantages importants — élasticité, paiement à l'usage, réduction du délai de mise en production — mais introduit également un changement de paradigme en matière de sécurité : la responsabilité est partagée entre le fournisseur cloud, qui sécurise l'infrastructure physique, et le client, qui reste responsable de la configuration de ses propres ressources, notamment de la gestion des identités et des accès.

1.3 AWS IAM et la problématique de la sécurité des identités

Le service *AWS Identity and Access Management* (IAM) permet de définir qui peut faire quoi sur un compte AWS. Il repose sur quelques concepts clés :

- les **utilisateurs** (*users*), qui représentent des identités individuelles ;
- les **groupes**, qui regroupent des utilisateurs partageant les mêmes permissions ;
- les **rôles**, qui peuvent être assumés temporairement par des utilisateurs ou des services ;
- les **politiques** (*policies*), documents JSON décrivant précisément les actions autorisées ou refusées sur quelles ressources.

Le principe de **moindre privilège** recommande de n'accorder à chaque identité que les permissions strictement nécessaires à l'exercice de sa fonction. En pratique, ce principe est souvent difficile à maintenir dans la durée : les permissions ont tendance à s'accumuler, les politiques trop permissives sont fréquentes, et les changements ne font pas toujours l'objet d'une revue systématique.

1.4 Limites et risques d'une mauvaise gestion IAM

Plusieurs scénarios illustrent les conséquences d'une gestion IAM déficiente ou d'une action malveillante :

- la création d'un utilisateur ou d'une clé d'accès non autorisée, pouvant servir de porte dérobée persistante ;
- l'attachement d'une politique excessive à un utilisateur ou un groupe, conduisant à une escalade de privilèges ;
- la désactivation de l'authentification multifacteur (MFA) sur un compte à privilèges, réduisant la robustesse de l'authentification ;
- la suppression d'une politique de sécurité existante, affaiblissant les contrôles en place.

Ces opérations, prises isolément, sont parfaitement légitimes dans de nombreux contextes d'administration. C'est bien leur combinaison avec un contexte suspect (adresse IP inhabituelle, horaire atypique, absence de justification) qui doit déclencher une vigilance accrue. Cela justifie la nécessité d'un système de supervision continu, capable de capter chaque changement IAM et de le qualifier automatiquement.

1.5 Besoin d'une solution de supervision

Face à ces constats, le besoin identifié est celui d'une solution :

- **automatisée**, ne nécessitant pas de revue manuelle systématique des journaux ;
- **en quasi temps réel**, pour permettre une réaction rapide en cas d'anomalie ;
- **peu coûteuse**, s'appuyant sur des services managés facturés à l'usage plutôt que sur une infrastructure dédiée ;
- **traçable**, conservant un historique exploitable de chaque événement traité ;
- **reproductible**, déployable de façon automatisée sur un nouveau compte AWS.

C'est cette analyse qui a conduit à la définition du projet `aws-iam-monitor`, détaillée dans le chapitre suivant.

Chapitre 2

Étude et conception de la solution

2.1 Problématique et objectifs

La problématique retenue pour le projet peut être reformulée de façon opérationnelle : *comment détecter automatiquement, classer selon leur criticité, et notifier les opérations IAM sensibles effectuées sur un compte AWS, tout en conservant un historique structuré de ces événements ?*

2.1.1 Objectifs fonctionnels

- détecter en temps quasi réel les opérations IAM sensibles (création d'utilisateur, suppression de politique, création de clé d'accès, désactivation de MFA, attachement de politique) ;
- classer chaque événement selon un niveau de risque (**CRITICAL**, **HIGH**, **MEDIUM**, **LOW**) ;
- vérifier l'adresse IP source de chaque appel d'API par rapport à une liste blanche configurable ;
- notifier les événements à risque élevé par e-mail via une alerte SNS ;
- archiver chaque événement traité sous forme d'enregistrement JSON structuré ;
- exposer des métriques exploitables pour la supervision.

2.1.2 Objectifs techniques

- s'appuyer exclusivement sur des services AWS managés, sans serveur à administrer (approche serverless) ;
- garantir l'idempotence du déploiement, qu'il soit rejoué plusieurs fois sans erreur ni duplication ;
- proposer deux implémentations indépendantes et équivalentes (Bash/AWS CLI et Terraform) ;
- documenter précisément l'architecture, l'installation et l'utilisation du projet.

2.2 Analyse des besoins et architecture générale

L'analyse du besoin a conduit à retenir une architecture événementielle en cinq étages, illustrée par le flux suivant :

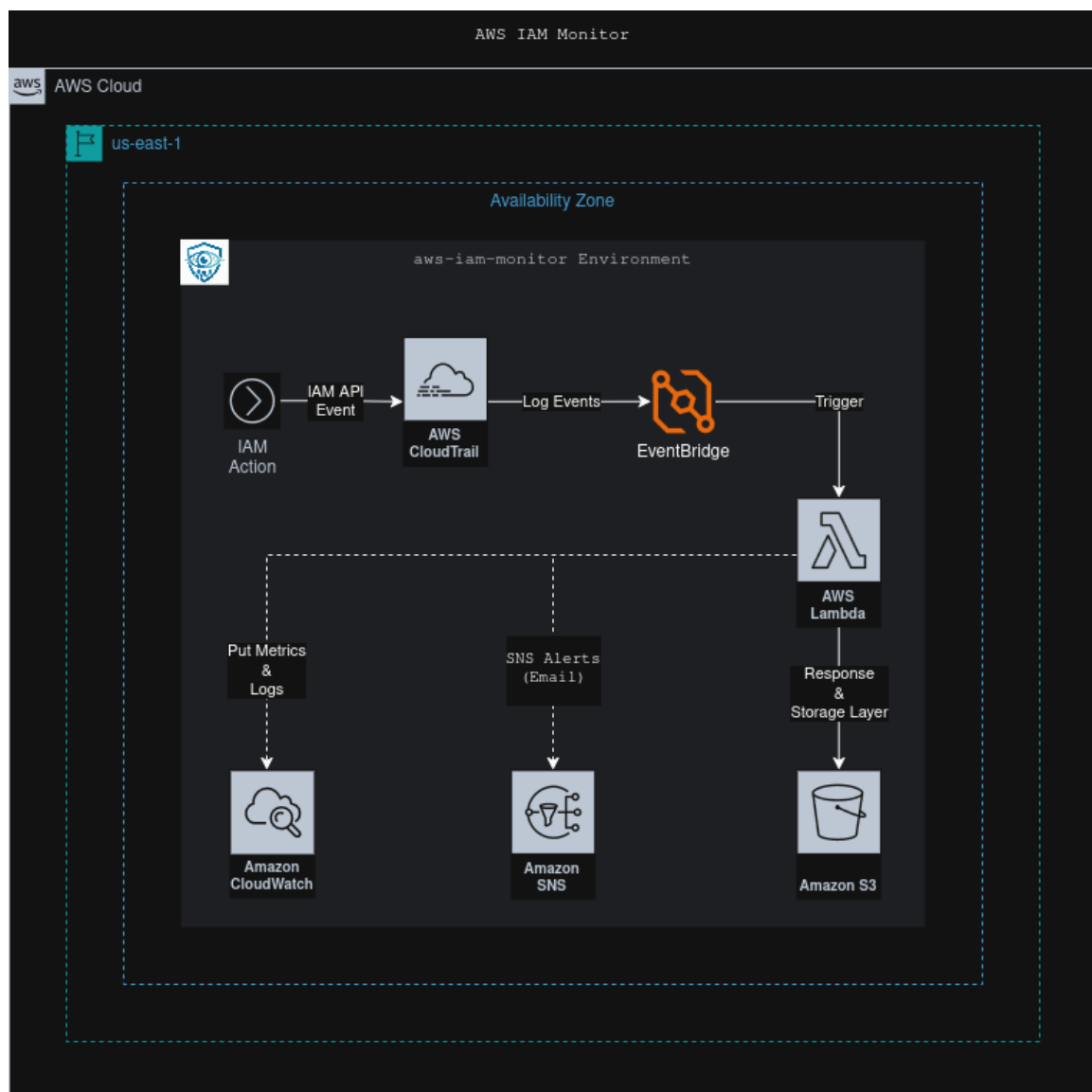


Figure 2.1: Schéma détaillé de l'architecture serverless du projet `aws-iam-monitor`, provisionnée à l'identique par les deux implémentations (Bash et Terraform).

Ce flux se décompose ainsi :

1. une opération IAM est effectuée sur le compte, qu'elle provienne d'un administrateur, d'un pipeline CI/CD ou d'une clé compromise ;

2. AWS CloudTrail enregistre cet appel d'API comme un événement de management, incluant l'identité appelante, l'action, la ressource ciblée et l'adresse IP source ;
3. Amazon EventBridge évalue cet événement au regard d'un *event pattern* ciblant la source `iam.amazonaws.com` et, si l'événement correspond, déclenche la fonction Lambda ;
4. la fonction Lambda, écrite en Python 3.11, analyse le payload reçu, vérifie l'adresse IP source par rapport à la liste blanche configurée, et classe l'événement selon son niveau de risque ;
5. selon le résultat de cette analyse, la fonction publie une alerte SNS pour les événements à risque élevé, archive systématiquement un enregistrement JSON structuré dans un bucket S3 organisé par date, et émet des métriques personnalisées vers CloudWatch.

2.3 Choix des services AWS

Service AWS	Rôle dans l'architecture
AWS IAM	Gestion des politiques, groupes, utilisateurs, rôles et de la politique de liste blanche IP.
AWS CloudTrail	Capture des événements de management (appels d'API) sur l'ensemble du compte.
Amazon EventBridge	Filtrage des événements CloudTrail selon un <i>event pattern</i> ciblant IAM et déclenchement de la Lambda.
AWS Lambda (Python 3.11)	Analyse du payload, vérification de l'IP source, classification du risque, archivage, alerte, métriques.
Amazon SNS	Diffusion des alertes de sécurité par e-mail pour les événements à risque élevé.
Amazon S3	Stockage chiffré (AES-256) et versionné des journaux CloudTrail et des enregistrements d'audit JSON.
Amazon CloudWatch	Journalisation, métriques personnalisées (<code>AWSIAMMonitor</code>) et, côté Terraform, alarmes.

Le choix d'une architecture entièrement serverless a été motivé par l'absence de charge opérationnelle liée à la gestion de serveurs, par une facturation à l'usage adaptée à un flux d'événements par nature irrégulier, et par la disponibilité native de l'intégration entre CloudTrail, EventBridge et Lambda.

2.4 Sécurité de l'architecture

Plusieurs choix de conception participent à la sécurité globale du système : le chiffrement systématique des buckets S3 (AES-256), le versioning activé sur ces mêmes buckets, l'attribution à la fonction Lambda d'un rôle d'exécution dont les permissions sont scopées aux ressources strictement nécessaires (particulièrement resserrées dans l'implémentation Terraform), ainsi que la vérification de l'adresse IP source de chaque opération IAM par rapport à une politique de liste blanche générée dynamiquement.

2.5 Classification des risques

La fonction Lambda applique une grille de classification à quatre niveaux (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) à chaque opération IAM interceptée. Cette classification tient compte à la fois de la nature de l'action réalisée (par exemple la suppression d'une politique ou la désactivation d'un

dispositif MFA constituent des signaux plus critiques que la simple consultation d'une ressource) et du contexte de l'appel, notamment la conformité de l'adresse IP source à la liste blanche configurée.

2.6 Stratégie de stockage et stratégie d'alerte

Chaque événement IAM traité par la Lambda, quel que soit son niveau de risque, est archivé sous la forme d'un enregistrement JSON structuré dans le bucket S3 d'audit, organisé selon une arborescence par date (YYYY/MM/DD/). Cette organisation permet une exploitation ultérieure simple, que ce soit pour une revue manuelle ou pour une éventuelle intégration avec un outil d'analyse externe.

La stratégie d'alerte, quant à elle, est volontairement sélective : seuls les événements dont le niveau de risque est jugé élevé déclenchent une publication sur le topic SNS `iam-alerts`, évitant ainsi la fatigue d'alerte (*alert fatigue*) qui résulterait d'une notification systématique de chaque opération IAM, y compris les plus anodines.

2.7 Infrastructure as Code : une conception à double implémentation

Un des choix structurants du projet a été de concevoir, dès la phase de design, une architecture pouvant être provisionnée de façon strictement équivalente par deux approches distinctes : une implémentation impérative en Bash/AWS CLI et une implémentation déclarative en Terraform. Ce choix a orienté la conception vers une séparation claire des responsabilités par composant AWS (IAM, CloudTrail, SNS, Lambda, EventBridge), chaque composant correspondant à un module Bash et à un module Terraform distincts, afin de faciliter la comparaison et la maintenance parallèle des deux implémentations.

Chapitre 3

Réalisation

Ce chapitre présente la réalisation technique du projet `aws-iam-monitor`, organisée par composant. Pour chaque composant, l'implémentation Bash et l'implémentation Terraform sont présentées conjointement, dans la mesure où elles ont été développées en miroir l'une de l'autre.

3.1 Structure générale du dépôt

Le dépôt du projet est organisé de la façon suivante :

```
aws-iam-monitor/
|-- Makefile, README.md, ARCHITECTURE.md
|-- INSTALL.md, USAGE.md, SECURITY.md
|-- project/
| |-- bash/ Implementation Bash / AWS CLI
| | |-- awsctl CLI d'orchestration principal
| | |-- lib/ Bibliotheque partagee (common, logger, requirements)
| | |-- iam/ Politiques, groupes, utilisateurs, whitelist IP
| | |-- cloud-trail/ Trail + bucket S3 de logs
| | |-- sns/ Topic d'alerte + abonnement email
| | |-- lambda/ Fonction de supervision + bucket d'audit
| | '-- event-bridge/ Regle + cible Lambda + permission
| '-- terraform/ Implementation Terraform (declarative)
|-- main.tf, variables.tf, outputs.tf
|-- providers.tf, versions.tf, locals.tf
|-- modules/ s3, iam, cloudtrail, sns, lambda,
| eventbridge, cloudwatch
|-- docapi/ Documentation Sphinx
'-- res/ Ressources (logos, schemas)
```

3.2 IAM

Le module IAM constitue la première brique déployée dans les deux implémentations. Il met en place un bac à sable de démonstration composé de politiques gérées (`DevPolicy`, `ProdPolicy`, `AuditPolicy`), de groupes (`DevGroup`, `ProdGroup`, `TestGroup`) et d'utilisateurs de démonstration (`DevAdmin`, `ProdAdmin`, etc.). La création de rôles IAM supplémentaires est optionnelle, activée respectivement par l'option `-r/-role` côté Bash ou par la variable `enable_iam_roles` côté Terraform.

Une politique `IPWhitelistPolicy` est générée dynamiquement à partir d'une liste d'adresses IP autorisées (`ALLOWED_IPS` côté Bash, `allowed_ips` côté Terraform), utilisée en aval par la fonction Lambda pour qualifier la provenance de chaque appel d'API.

Côté Terraform, ce module est entièrement autonome, sans dépendance croisée avec les autres modules, ce qui facilite sa maintenance indépendante.

3.3 CloudTrail

Le module CloudTrail provisionne un bucket S3 dédié à la réception des journaux (chiffré en AES-256, versionné) ainsi qu'un *trail* capturant l'ensemble des événements de management du compte, avec un type de lecture/écriture couvrant l'intégralité des événements (A11). Côté Terraform, ce module réutilise une instance du module générique `s3` et lui associe la politique de bucket spécifique à CloudTrail.

3.4 EventBridge

Le module EventBridge définit la règle qui filtre, parmi l'ensemble des événements capturés par CloudTrail, ceux dont la source est `iam.amazonaws.com`. Cette règle cible la fonction Lambda de supervision et s'accompagne de l'attribution de la permission d'invocation nécessaire pour qu'EventBridge puisse déclencher la fonction. Le filtrage fin des actions réellement surveillées (liste des opérations sensibles) est délégué à la logique applicative interne de la Lambda plutôt qu'à l'*event pattern* lui-même, afin de conserver une règle de filtrage simple et une logique métier centralisée et facilement évolutive.

3.5 Lambda

La fonction Lambda, écrite en Python 3.11, constitue le cœur analytique du système. Son traitement se décompose en plusieurs étapes :

1. réception et analyse du payload de l'événement transmis par EventBridge ;
2. extraction des champs pertinents : identité appelante, action réalisée, ressource ciblée, adresse IP source ;
3. vérification de la conformité de l'adresse IP source par rapport à la politique de liste blanche ;
4. classification du niveau de risque de l'opération (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) ;
5. archivage systématique d'un enregistrement JSON structuré dans le bucket S3 d'audit, organisé par date ;
6. publication d'une alerte SNS pour les événements classés à risque élevé ;
7. émission de métriques personnalisées vers CloudWatch, sous le namespace `AWSIAMMonitor`.

Le code source de la fonction est partagé entre les deux implémentations (`project/bash/lambda/src/lambda_function.py`), ce qui garantit une logique de détection strictement identique quelle que soit l'implémentation utilisée pour provisionner l'infrastructure. Côté Terraform, le déploiement du code est automatisé via l'attribut `source_code_hash`, qui détecte toute modification du fichier source et déclenche un redéploiement sans étape de packaging manuelle ; côté Bash, cette étape est réalisée par une comparaison manuelle de hachage SHA-256 avant mise à jour.

3.6 SNS

Le module SNS provisionne le topic d'alerte de sécurité `iam-alerts`, sa politique d'accès, ainsi qu'un abonnement e-mail optionnel, configuré via la variable `SNS_ALERT_EMAIL` (fichier `.env`) côté Bash, ou via la variable Terraform `sns_alert_email`.

3.7 S3

Deux buckets S3 distincts sont utilisés par le système : un bucket recevant les journaux bruts de CloudTrail, et un bucket dédié à l'historique d'audit alimenté par la fonction Lambda. Les deux sont chiffrés (algorithme configurable, **AES256** par défaut) et versionnés. Côté Terraform, un module **s3** générique et réutilisable a été développé, ne portant aucune politique de bucket propre — celle-ci étant définie par le module appelant (**cloudtrail** ou **lambda**) — et complété par un blocage explicite de l'accès public sur les deux buckets, amélioration propre à l'implémentation Terraform par rapport à l'implémentation Bash.

3.8 CloudWatch

Le module CloudWatch gère le groupe de logs de la fonction Lambda (durée de rétention configurable, 30 jours par défaut) ainsi que les métriques personnalisées émises par la Lambda. L'implémentation Terraform ajoute une fonctionnalité absente de l'implémentation Bash : des alarmes optionnelles (activées par défaut via la variable **enable_cloudwatch_alarms**) portant sur les accès depuis une IP non autorisée (**UnauthorizedIPAccess**) et sur les erreurs de la fonction Lambda, ces alarmes publiant elles-mêmes vers le topic SNS existant.

3.9 Bash / AWS CLI – l'outil **awsctl**

L'implémentation Bash est pilotée par un unique script d'orchestration, **awsctl**, dont la syntaxe générale est :

```
./awsctl <ACTION> [TARGET] [OPTIONS]
```

Les actions disponibles sont **-up** (création), **-down** (destruction), **-status** (état) et **-test** (exécution des suites de test). Les cibles possibles correspondent à chaque composant (**all**, **iam**, **cloud-trail**, **sns**, **lambda**, **event-bridge**), ce qui permet de piloter l'infrastructure globalement ou composant par composant. Les options **-r/-role** et **-v/-verbose** complètent cette interface.

L'ensemble des scripts s'appuie sur une bibliothèque commune (**lib/**), regroupant les fonctions partagées, la journalisation et la vérification des prérequis, ce qui limite la duplication de code entre les modules. Un soin particulier a été apporté à l'**idempotence** des scripts : chaque opération de création vérifie préalablement l'existence de la ressource visée, afin qu'un déploiement rejoué plusieurs fois ne produise ni erreur ni doublon.

Le déploiement complet suit un ordre explicite en cinq étapes : IAM, puis CloudTrail, puis SNS, puis Lambda, puis EventBridge — la teardown appliquant l'ordre inverse. Cet enchaînement gère notamment la propagation IAM parfois différée d'AWS lors de la création de la fonction Lambda, au moyen d'une boucle de nouvelles tentatives avec temporisation.

3.10 Terraform

L'implémentation Terraform reproduit la même architecture de façon déclarative, organisée en sept modules réutilisables : **s3**, **cloudtrail**, **iam**, **sns**, **cloudwatch**, **lambda** et **eventbridge**. Le fichier racine **main.tf** assure le câblage entre ces modules, dont l'ordre de résolution est déterminé automatiquement par le graphe de dépendances Terraform, sans qu'il soit nécessaire de l'explicitier comme en Bash.

L'ensemble de la configuration est piloté par variables (**variables.tf**, valeurs par défaut alignées sur celles du fichier **.env.example** de l'implémentation Bash), ce qui garantit qu'un simple **terraform apply** sans surcharge produit une architecture équivalente à **./awsctl -up all**. Terraform ne lit pas le fichier **.env** de l'implémentation Bash : la configuration se fait exclusivement via **terraform.tfvars**.

3.11 Comparaison Bash / Terraform

Bien que fonctionnellement équivalentes, les deux implémentations présentent des différences techniques volontaires, résumées dans le tableau suivant.

Aspect	Bash / AWS CLI	Terraform
Approche	Impérative (séquence explicite de commandes)	Déclarative (état cible décrit, ordre résolu automatiquement)
Gestion des permissions Lambda	Portée large (<code>S3AuditHistory</code> , <code>Resource: "*" pour SNS</code>)	Portée strictement limitée aux ressources exactes (moindre privilège renforcé)
Accès public S3	Non explicitement bloqué	Bloqué explicitement sur les deux buckets
Attente de propagation IAM	Boucle de 6 tentatives avec délai de 10s	Temporisation déterministe unique (<code>time_sleep</code> , 10s)
Déploiement du code Lambda	Comparaison manuelle de hachage SHA-256	Automatique via <code>source_code_hash</code>
Alarmes CloudWatch	Absentes (métriques émises mais non exploitées)	Optionnelles, activées par défaut
Reproductibilité	Élevée mais dépendante de l'ordre d'exécution des scripts	Élevée et garantie par le graphe de dépendances
Maintenance	Simple pour de petites évolutions ciblées	Facilitée par la modularité et l'état déclaratif pour des évolutions complexes

En dehors de ces différences assumées, le nommage des ressources, le contenu de l'*event pattern* EventBridge, les documents de politique IAM (`DevPolicy`, `ProdPolicy`, `AuditPolicy`, `IPWhitelistPolicy`) et la politique du topic SNS restent strictement identiques entre les deux implémentations.

3.12 Tests et validation

La validation du projet s'est appuyée sur les mécanismes fournis par le dépôt lui-même : la commande `./awsctl -test all` exécute la suite de tests associée à l'implémentation Bash, tandis que `terraform validate` et `terraform plan` permettent de vérifier la cohérence de la configuration Terraform avant tout déploiement réel. Le cycle complet de provisionnement (`-up all / terraform apply`), de vérification (`-status all / consultation des outputs Terraform`) et de désinstallation (`-down all / terraform destroy`) a été exécuté de bout en bout sur les deux implémentations.

Chapitre 4

Déploiement, validation et résultats

4.1 Prérequis

Le déploiement du projet, quelle que soit l'implémentation choisie, nécessite un compte AWS actif, des identifiants disposant des permissions nécessaires à la création des ressources IAM, CloudTrail, Lambda, EventBridge, SNS, S3 et CloudWatch, ainsi qu'AWS CLI configuré (résolution standard des identifiants : variables d'environnement, fichier de configuration partagé, profil SSO, ou rôle attaché). L'implémentation Terraform nécessite en outre l'installation de Terraform, tandis que l'implémentation Bash nécessite un environnement Unix disposant de Bash et des utilitaires standards.

4.2 Installation et provisioning

4.2.1 Implémentation Bash

```
git clone https://github.com/ZouariOmar/aws-iam-monitor.git
cd aws-iam-monitor
cp project/bash/.env.example project/bash/.env
# Renseigner SNS_ALERT_EMAIL et ALLOWED_IPS si nécessaire
make bash-up
```

4.2.2 Implémentation Terraform

```
git clone https://github.com/ZouariOmar/aws-iam-monitor.git
cd aws-iam-monitor
cp project/terraform/terraform.tfvars.example project/terraform/terraform.tfvars
make tf-init
make tf-apply
```

Avertissement : les deux implémentations utilisant par défaut les mêmes noms de ressources, elles ne doivent pas être déployées simultanément sur le même compte AWS, sous peine de conflit de nommage.

4.3 Déroulement d'un déploiement complet

Un déploiement complet (`./awsctl -up all` ou `terraform apply` avec les valeurs par défaut) provisionne, dans l'ordre : les politiques, groupes et utilisateurs IAM de démonstration ; le bucket de logs et le trail CloudTrail ; le topic SNS et son éventuel abonnement e-mail ; le bucket d'audit et la fonction Lambda avec son rôle d'exécution ; puis la règle EventBridge et sa cible. Côté Terraform, ce même résultat est obtenu sans que l'ordre soit explicité par l'opérateur, celui-ci étant déduit du graphe de dépendances entre modules.

4.4 Fonctionnement observé

Une fois déployé, le système traite chaque opération IAM effectuée sur le compte de la façon suivante : l'événement est capturé par CloudTrail, filtré par la règle EventBridge, puis transmis à la fonction Lambda qui l'analyse, le classe, l'archive dans le bucket d'audit sous forme de fichier JSON horodaté, et déclenche le cas échéant une alerte SNS ainsi que l'émission d'une métrique CloudWatch. Ce traitement s'effectue en l'espace de quelques secondes après l'opération IAM initiale, conformément à l'objectif de détection en temps quasi réel fixé en début de projet.

4.4.1 Illustrations

Les captures d'écran suivantes, réalisées sur le compte AWS de test, illustrent le fonctionnement effectif du système une fois déployé.



Figure 4.1: Confirmation d'abonnement au topic SNS `iam-alerts` reçue par e-mail lors de la configuration du canal de notification.

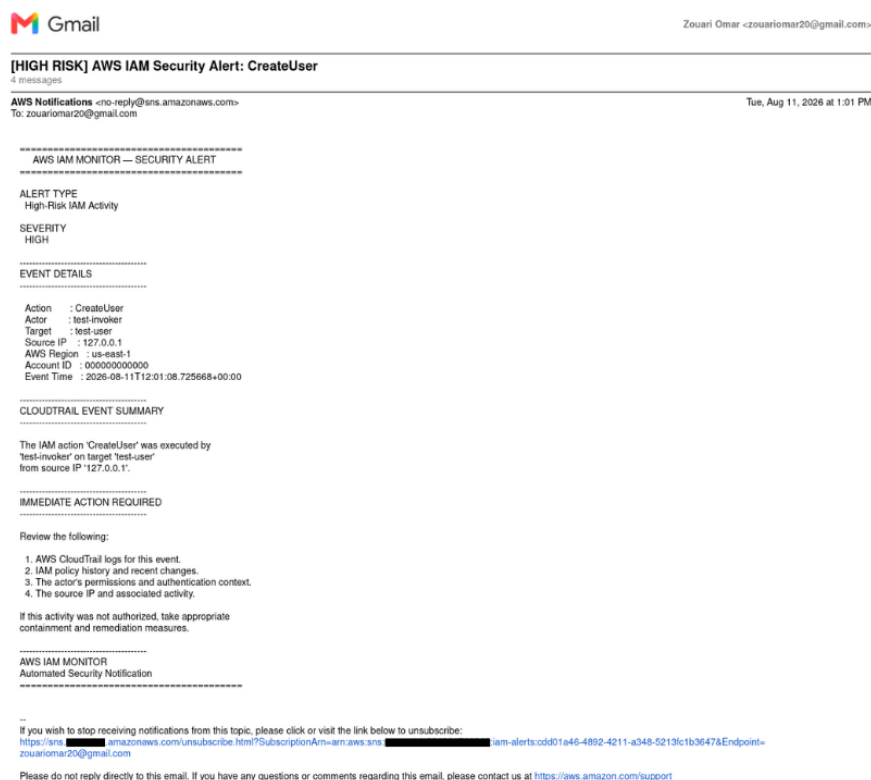


Figure 4.2: Alerte SNS reçue pour un événement classé à risque élevé : création d'un utilisateur IAM (`CreateUser`).

4.5 Avantages de l'approche idempotente

L'idempotence recherchée dans les deux implémentations présente un intérêt opérationnel concret : un même script ou une même configuration peut être exécuté à plusieurs reprises sans provoquer d'erreur ni de duplication de ressources, ce qui facilite les itérations de développe-

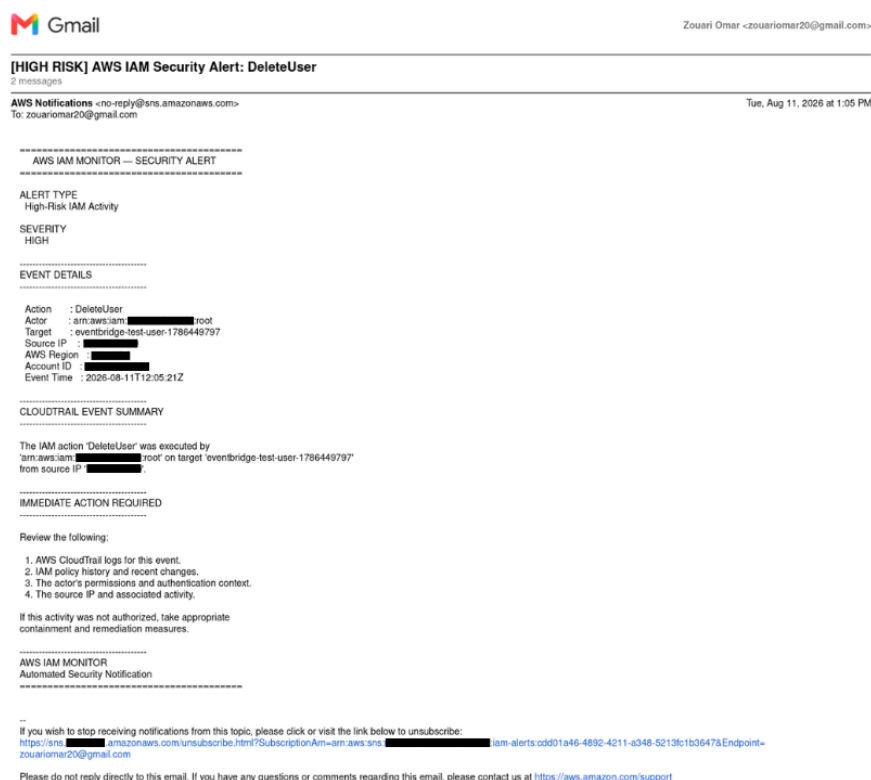


Figure 4.3: Alerte SNS reçue pour un événement classé à risque élevé : suppression d'un utilisateur IAM (DeleteUser).

ment, les corrections successives et la reproductibilité du déploiement sur un nouveau compte AWS.

4.6 Limites de la validation effectuée

La validation réalisée durant le stage a porté sur le fonctionnement correct du cycle de déploiement, de vérification et de désinstallation des deux implémentations, ainsi que sur la cohérence de l'architecture obtenue par rapport à la conception initiale. Elle ne constitue toutefois pas une campagne de test de charge ni un audit de sécurité formel ; ces aspects relèvent des perspectives évoquées au chapitre suivant.

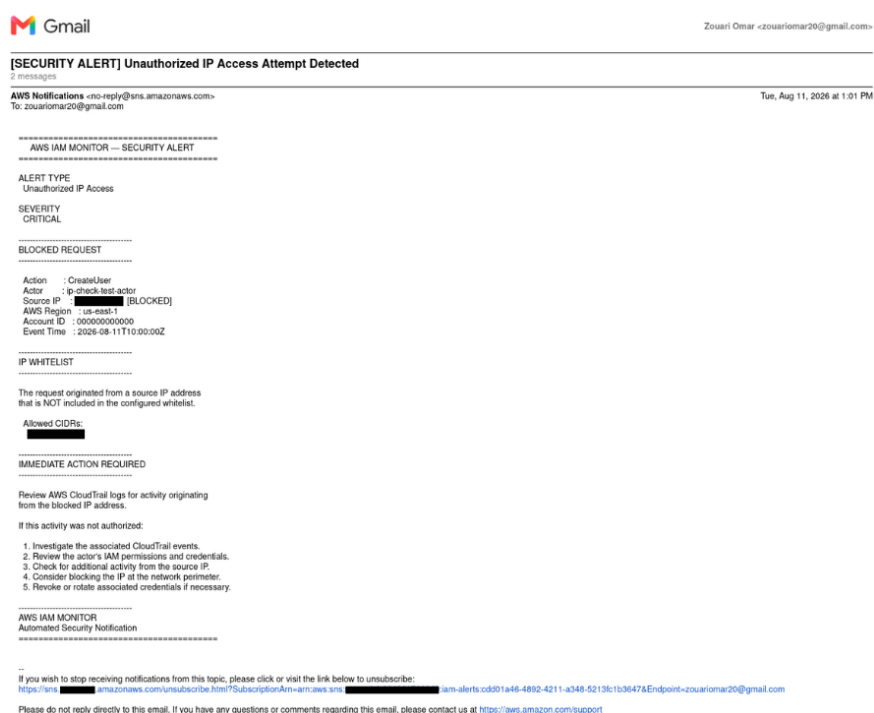


Figure 4.4: Alerte SNS déclenchée à la suite d'une opération IAM provenant d'une adresse IP absente de la liste blanche configurée.

Chapitre 5

Bilan et perspectives

5.1 Compétences techniques acquises

Ce stage a permis de développer un ensemble de compétences techniques concrètes, structurées autour de plusieurs axes.

5.1.1 Compétences en Cloud AWS et sécurité

La manipulation quotidienne des services IAM, CloudTrail, EventBridge, Lambda, SNS, S3 et CloudWatch a permis d'acquérir une compréhension pratique, et non plus seulement théorique, de la manière dont ces services interagissent pour construire une architecture de sécurité serverless cohérente. La réflexion menée sur la classification des risques et la logique de liste blanche IP a par ailleurs sensibilisé à la démarche d'analyse de risque propre à la sécurité des identités.

5.1.2 Compétences DevOps et Infrastructure as Code

Le développement en parallèle d'une implémentation impérative (Bash) et d'une implémentation déclarative (Terraform) a permis de comparer concrètement ces deux paradigmes d'automatisation d'infrastructure, chacun avec ses forces (simplicité et contrôle fin pour Bash, robustesse et déclarativité pour Terraform).

5.1.3 Compétences en développement

Le développement de la fonction Lambda en Python a mobilisé des compétences de traitement de données structurées (JSON), de logique conditionnelle de classification, et d'intégration avec plusieurs services AWS via leurs SDK respectifs. Le scripting Bash a quant à lui renforcé la rigueur nécessaire à l'écriture de scripts idempotents et modulaires.

5.1.4 Documentation et autonomie

La rédaction de la documentation technique du projet (architecture, installation, usage) a constitué un exercice à part entière, nécessitant de formaliser clairement des choix techniques pour un lecteur qui n'a pas participé à leur élaboration. Le stage a par ailleurs été mené avec un degré d'autonomie important dans la planification du travail au sein du cadre fixé.

5.2 Difficultés rencontrées

Certaines difficultés inhérentes à ce type de projet peuvent être identifiées à partir de l'architecture elle-même, sans qu'il soit possible d'en attribuer précisément la résolution à un événement daté faute de trace exhaustive de chaque incident : la propagation parfois différée des permissions IAM lors de la création d'une fonction Lambda a nécessité la mise en place de mécanismes d'attente (boucle de nouvelles tentatives côté Bash, temporisation déterministe côté Terraform) ; la nécessité de maintenir une stricte équivalence fonctionnelle entre deux implémentations développées en parallèle a également demandé une vigilance constante afin d'éviter toute divergence non intentionnelle.

5.3 Limites actuelles du projet

Le projet, dans son état actuel, présente certaines limites assumées : la classification des risques repose sur une logique embarquée dans le code de la Lambda plutôt que sur un moteur de règles configurable ; le système ne couvre que les événements de management IAM capturés par CloudTrail, à l'exclusion d'autres sources de télémétrie de sécurité ; et l'état Terraform, en configuration par défaut, repose sur un backend local peu adapté à un usage en équipe.

5.4 Perspectives d'évolution

Plusieurs pistes d'amélioration se dégagent naturellement de ce travail, à titre de perspectives futures et non de fonctionnalités déjà réalisées :

- enrichissement de la logique de classification des risques, par exemple via un moteur de règles externalisé et configurable sans modification du code de la Lambda ;
- extension de la couverture événementielle au-delà des seules opérations IAM, vers d'autres services sensibles ;
- intégration avec des outils de sécurité tiers (SIEM, plateformes de gestion des incidents) pour centraliser les alertes ;
- mise en place de tableaux de bord CloudWatch plus avancés, exploitant plus largement les métriques `AWSIAMMonitor` déjà émises ;
- prise en charge du support multi-comptes via AWS Organizations, afin de superviser plusieurs comptes AWS depuis un point central ;
- automatisation de réponses à certains incidents (par exemple désactivation automatique d'une clé d'accès jugée compromise), avec les précautions nécessaires pour éviter les faux positifs ;
- mise en place d'un backend Terraform distant (S3 + verrouillage DynamoDB) pour sécuriser l'état en configuration d'équipe.

Conclusion générale

Ce stage de deux mois a permis de mener à bien un projet complet de sécurité cloud, depuis l'apprentissage initial des services AWS impliqués jusqu'à la livraison d'une plateforme de supervision IAM fonctionnelle, documentée et déployable de deux façons équivalentes.

La première semaine, consacrée à la découverte d'AWS IAM, CloudTrail, EventBridge, Lambda, SNS, S3 et CloudWatch, a posé les bases nécessaires à la conception d'une architecture serverless événementielle cohérente. La phase de développement, qui a occupé l'essentiel du stage, a permis de construire progressivement chaque composant de cette architecture, d'abord dans une implémentation Bash/AWS CLI orchestrée par l'outil `awsctl`, puis dans une implémentation Terraform déclarative et modulaire, strictement équivalente sur le plan fonctionnel. Cette double implémentation a constitué l'un des apports pédagogiques les plus riches du stage, en permettant une comparaison directe et concrète entre deux approches d'automatisation d'infrastructure.

Sur le plan des compétences, ce stage a renforcé une compréhension opérationnelle de la sécurité des identités dans le cloud, une maîtrise pratique du scripting Bash et du développement Python appliqué au traitement d'événements, ainsi qu'une première expérience approfondie de l'infrastructure as code avec Terraform. La nécessité de documenter précisément chaque choix technique a par ailleurs développé une rigueur de communication technique indispensable à tout projet destiné à être repris ou audité par des tiers.

Au-delà du résultat technique obtenu, ce travail illustre une démarche de sécurité proactive : plutôt que de constater a posteriori les conséquences d'une mauvaise configuration IAM, le projet `aws-iam-monitor` propose un mécanisme de détection continue, capable d'alerter rapidement sur les changements les plus sensibles. Les perspectives identifiées, qu'il s'agisse de l'enrichissement de la classification des risques, du support multi-comptes ou de l'intégration avec des outils de sécurité tiers, dessinent des axes de travail réalistes pour la poursuite de ce projet au-delà du cadre du stage.

Points clés pour la soutenance

Problématique

Comment détecter, classer et notifier automatiquement les modifications sensibles apportées à AWS IAM sur un compte, en temps quasi réel et à moindre coût opérationnel ?

Solution

`aws-iam-monitor` : plateforme serverless événementielle, développée en deux implémentations équivalentes (Bash/AWS CLI et Terraform).

Architecture

IAM → CloudTrail → EventBridge → Lambda (analyse, classification, whitelist IP) → SNS + S3 + CloudWatch.

Services AWS mobilisés

IAM, CloudTrail, EventBridge, Lambda (Python 3.11), SNS, S3, CloudWatch.

Démonstration possible

Déclenchement d'une opération IAM sensible (ex. `CreateAccessKey`) → observation de l'archive JSON dans S3, de l'alerte SNS et de la métrique CloudWatch correspondantes.

Choix techniques marquants

Idempotence des scripts Bash ; modularité et graphe de dépendances Terraform ; code Lambda partagé entre les deux implémentations via `source_code_hash`.

Sécurité

Chiffrement et versioning S3, blocage de l'accès public (Terraform), permissions Lambda scopées au strict nécessaire, liste blanche IP générée dynamiquement.

Bash vs Terraform

Deux implémentations indépendantes, fonctionnellement équivalentes, avec des divergences assumées et documentées (permissions, alarmes CloudWatch, gestion du déploiement du code).

Difficultés

Propagation IAM différée lors de la création de la Lambda ; maintien de la stricte équivalence entre les deux implémentations développées en parallèle.

Résultats

Cycle complet de provisionnement, vérification et désinstallation validé sur les deux implémentations ; documentation technique complète du projet.

Limites

Classification des risques embarquée dans le code plutôt que configurable ; périmètre limité aux événements IAM ; backend Terraform local par défaut.

Perspectives

Moteur de classification configurable, support multi-comptes AWS Organizations, intégration SIEM, tableaux de bord avancés, backend Terraform distant.